

Введение в работу с данными

Лабораторная работа № 7

Шулуужук А. В.

25 ноябрь 2025

Российский университет дружбы народов, Москва, Россия

Основной целью работы является специализированных пакетов Julia для обработки данных.

Выполнение лабораторной работы

```
[6]: using CSV, DataFrames, DelimitedFiles
```

```
[8]: # Считывание данных и их запись в структуру:  
P = CSV.File("programminglanguages.csv") |> DataFrame
```

```
[8]: 73×2 DataFrame
```

	Row	year	language
	Int64	String31	
	1	1951	Regional Assembly Language
	2	1952	Autocode
	3	1954	IPL
	4	1955	FLOW-MATIC
	5	1957	FORTRAN

Рис. 1: Считывание данных и их запись в структуру

```
# Функция определения по названию языка программирования года его создания:  
function language_created_year(P,language::String)  
    loc = findfirst(P[:,2].==language)  
    return P[loc,1]  
end  
# Пример вызова функции и определение даты создания языка Python:  
language_created_year(P,"Python")
```

1991

```
# Пример вызова функции и определение даты создания языка Julia:  
language_created_year(P,"Julia")
```

2012

Рис. 2: Функция определения по названию языка программирования года его создания

Запись данных в файл

```
# Запись данных в CSV-файл:
CSV.write("programming_languages_data2.csv", P)

"programming_languages_data2.csv"

# Пример записи данных в текстовый файл с разделителем ',':
writedlm("programming_languages_data.txt", Tx, ',')

# Пример записи данных в текстовый файл с разделителем '-':
writedlm("programming_languages_data2.txt", Tx, '-')

# Построчное считывание данных с указанием разделителя:
P_new_delim = readdlm("programming_languages_data2.txt", '-')

74x2 Matrix{Any}:
  "year"  "language"
1951     "Regional Assembly Language"
1952     "Autocode"
1954     "IPL"
1955     "FLOW-MATIC"
1957     "FORTRAN"
1957     "COMTRAN"
1958     "LISP"
1958     "ALGOL 58"
1959     "FACT"
1959     "COBOL"
1959     "RPG"
1962     "APL"
```

Рис. 3: Построчное считывание данных с указанием разделителя

3. Словари

```
# Инициализация словаря:  
dict = Dict{Integer,Vector{String}}{}
```

```
Dict{Integer, Vector{String}}{}
```

```
# Инициализация словаря:  
dict2 = Dict{}
```

```
Dict{Any, Any}()
```

```
# Заполнение словаря данными:  
for i = 1:size(P,1)  
    year,lang = P[i,:]  
    if year in keys(dict)  
        dict[year] = push!(dict[year],lang)  
    else  
        dict[year] = [lang]  
    end  
end
```

```
# Пример определения в словаре языков программирования, созданных в 2003 году:  
dict[2003]
```

```
2-element Vector{String}:  
 "Groovy"  
 "Scala"
```

Рис. 4: Словари

4. DataFrames

```
: # Подгружаем пакет DataFrames:  
using DataFrames  
  
: # Задаём переменную со структурой DataFrame:  
df = DataFrame(year = P[:,1], language = P[:,2])
```

: 73×2 DataFrame

Row	year	language
	Int64	String31
1	1951	Regional Assembly Language
2	1952	Autocode
3	1954	IPL
4	1955	FLOW-MATIC
5	1957	FORTRAN

Рис. 5: Использование пакета DataFrames

```
# Получение статистических сведений о фрейме:  
describe(df)
```

2×7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	year	1982.99	1951	1986.0	2014	0	Int64
2	language		ALGOL 58		dBase III	0	String31

Рис. 6: Использование пакета DataFrames

5. RDatasets

```
#import Pkg
#Pkg.add("RDatasets")
# Подгружаем пакет RDatasets:
using RDatasets
```

```
# Задаём структуру данных в виде набора данных:
iris = dataset("datasets", "iris")
```

150×5 DataFrame

Row	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
	Float64	Float64	Float64	Float64	Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa

Рис. 7: Использование пакета RDatasets

```
# Определения типа переменной:
typeof(iris)
```

DataFrame

```
# Пакет RDatasets также предоставляет возможность с помощью description получить основн
describe(iris)
```

5×7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	SepalLength	5.84333	4.3	5.8	7.9	0	Float64
2	SepalWidth	3.05733	2.0	3.0	4.4	0	Float64
3	PetalLength	3.758	1.0	4.35	6.9	0	Float64
4	PetalWidth	1.19933	0.1	1.3	2.5	0	Float64
5	Species		setosa		virginica	0	CategoricalValue{String, UInt8}

Рис. 8: Использование пакета RDatasets

Работа с переменными отсутствующего типа (Missing Values)

```
# Отсутствующий тип:
a = missing
typeof(a)

Missing

# Пример операции с переменной отсутствующего типа:
a + 1

missing

# Определение перечня продуктов:
foods = ["apple", "cucumber", "tomato", "banana"]
# Определение калорий:
calories = [missing, 47, 22, 105]

4-element Vector{Union{Missing, Int64}}:
 missing
    47
    22
   105

# Определение типа переменной:
typeof(calories)

Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})
```

Рис. 9: Работа с переменными отсутствующего типа

Работа с переменными отсутствующего типа (Missing Values)

```
using Statistics

# Определение среднего значения:
mean(calories)

missing

# Определение среднего значения без значений с отсутствующим типом:
mean(skipmissing(calories))

58.0

# Задание сведений о ценах:
prices = [0.85, 1.6, 0.8, 0.6]

4-element Vector{Float64}:
 0.85
 1.6
 0.8
 0.6

# Формирование данных о калориях:
dataframe_calories = DataFrame(item=foods, calories=calories)

4×2 DataFrame
Row  item      calories
String  Int64?
1  apple    missing
2  cucumber    47
3  tomato     22
4  banana     105
```

Рис. 10: Работа с пакетом Statistics

Кластеризация данных. Метод k-средних

```
using DataFrames
using CSV

# Загрузка данных:
houses = CSV.File("houses.csv") |> DataFrame
```

985×12 DataFrame 960 rows omitted

Row	street	city	zip	state	beds	baths	sq_ft	type	sale_date	price
	String	String15	Int64	String3	Int64	Int64	Int64	String15	String31	Int64
1	3526 HIGH ST	SACRAMENTO	95838	CA	2	1	836	Residential	Wed May 21 00:00:00 EDT 2008	592,000
2	51 OMAHA CT	SACRAMENTO	95823	CA	3	1	1167	Residential	Wed May 21 00:00:00 EDT 2008	682,000
3	2796 BRANCH ST	SACRAMENTO	95815	CA	2	1	796	Residential	Wed May 21 00:00:00 EDT 2008	688,000
4	2805 JANETTE WAY	SACRAMENTO	95815	CA	2	1	852	Residential	Wed May 21 00:00:00 EDT 2008	693,000
5	6001 MCMAHON DR	SACRAMENTO	95824	CA	2	1	797	Residential	Wed May 21 00:00:00 EDT 2008	819,000

Рис. 11: Загрузка пакетов и необходимых данных

Кластеризация данных. Метод k-средних

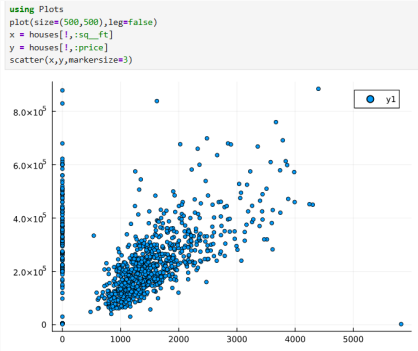


Рис. 12: График цен на недвижимость в зависимости от площади

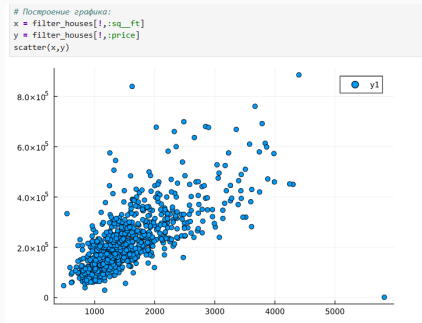


Рис. 13: График цен на недвижимость в зависимости от площади

Кластеризация данных. Метод k-средних

```
using DataFrames, Statistics
# Определение средней цены для определённого типа домов:
res = combine(groupby(filter_houses, :type), :price => mean => :mean_price)
```

3×2 DataFrame

Row	type	mean_price
	String15	Float64
1	Residential	2.34802e5
2	Condo	1.34213e5
3	Multi-Family	2.24535e5

4

```
using Clustering
# Добавление данных :latitude и :longitude в новый фрейм:
X = filter_houses[:, [:latitude, :longitude]]
# Конвертация данных в матричный вид:
X_matrix = Matrix(X)
```

814×2 Matrix{Float64}:
38.6319 -121.435
38.4789 -121.431
38.6183 -121.444
38.6168 -121.439
38.5195 -121.436

Рис. 14: Подключение пакета Statistic

Кластеризация данных. Метод k-средних

```
# Транспонирование матрицы с данными:
X_matrix = X_matrix'
```

```
2x814 adjoint(::Matrix{Float64}) with eltype Float64:
 38.6319  38.4789  38.6183  ...  38.7088  38.417  38.6552
-121.435 -121.431 -121.444  ... -121.257 -121.397 -121.076
```

```
# Задание количества кластеров:
k = length(unique(filter_houses[:,zip]))

66
```

```
# Определение k-среднего:
C = kmeans(X_matrix,k)
```

```
KmeansResult{Matrix{Float64}, Float64, Int64}([38.597473238769225 38.95380275 ... 38.56448544444
444 38.46755333333334; -121.40497961538459 -121.06692125000001 ... -121.32383233333333 -121.3178
6866666668], [39, 23, 39, 39, 53, 64, 16, 24, 28, 32 ... 19, 45, 23, 18, 52, 3, 50, 14, 41, 3
3], [9.283252802561037e-5, 0.00044116201024735346, 5.180126754567027e-5, 7.710813224548474e-5,
0.00018491080481908284, 0.00016102628069347702, 0.00026847111075767316, 0.0002836940730048809,
0.0004919182101730257, 0.00011686658399412408 ... 3.845047467621043e-5, 5.2413251978578046e-5,
9.496071652392857e-5, 0.00023885654809419066, 0.00013415837747743353, 0.00010097602716996334,
8.79761173564475e-5, 0.0005586109509749804, 0.0003121271583950147, 0.0006656794284936041], [1
3, 4, 24, 5, 20, 10, 17, 3, 28, 4 ... 4, 4, 8, 9, 10, 14, 3, 16, 9, 3], [13, 4, 24, 5, 20, 10,
17, 3, 28, 4 ... 4, 4, 8, 9, 10, 14, 3, 16, 9, 3], 0.19843561706875334, 9, true)
```

```
# Формирование фрейма данных:
df = DataFrame(cluster = C.assignments,city = filter_houses[:,city],
latitude = filter_houses[:,latitude],longitude = filter_houses[:,longitude],zip = filter_hou
es[:,zip])
```

Рис. 15: Подключение пакета Statistic

Кластеризация данных. Метод k-средних

```
clusters_figure = plot(legend = false)
for i = 1:k
    clustered_houses = df[df[:,cluster].== i,:]
    xvals = clustered_houses[:,latitude]
    yvals = clustered_houses[:,longitude]
    scatter!(clusters_figure,xvals,yvals,markersize=4)
end
xlabel!("Latitude")
ylabel!("Longitude")
title!("Houses color-coded by cluster")
display(clusters_figure)
```

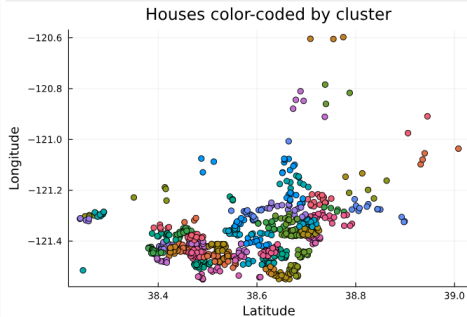


Рис. 16: Пример кластеризации объектов недвижимости по географическому расположению

Кластеризация данных. Метод k-средних

```
unique_zips = unique(filter_houses[:,zip])
zips_figure = plot(legend = false)
for uzip in unique_zips
    subs = filter_houses[filter_houses[:,zip].==uzip,:]
    x = subs[:,latitude]
    y = subs[:,longitude]
    scatter!(zips_figure,x,y)
end
xlabel!("Latitude")
ylabel!("Longitude")
title!("Houses color-coded by zip code")
display(zips_figure)
```

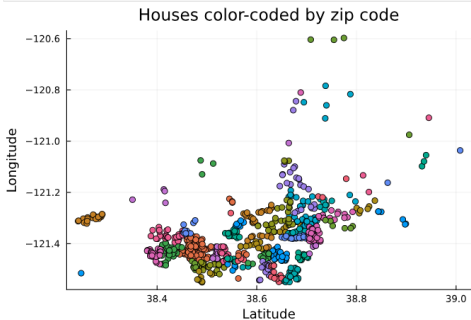


Рис. 17: Пример кластеризации объектов недвижимости по почтовому индексу

Кластеризация данных. Метод k ближайших соседей

```
using NearestNeighbors

knearest = 10
id = 70
point = X_matrix[:,id]

2-element Vector{Float64}:
 38.44004
-121.421012

# Поиск ближайших соседей:
kdtree = KDTree(X_matrix)
idxs, dists = knn(kdtree, point, knearest, true)

([70, 764, 196, 125, 557, 368, 415, 92, 112, 683], [0.0, 0.006264891539364138, 0.00825320259050462, 0.0084735851326300
5065124697706, 0.009921759722950759, 0.009941028618812013, 0.010332637707777167, 0.011168993911721985])
```

Рис. 18: Загрузка пакетов и необходимых данных

Кластеризация данных. Метод k ближайших соседей

```
# Все объекты недвижимости:  
x = filter_houses[:, :latitude];  
y = filter_houses[:, :longitude];  
scatter(x, y)  
  
# Соседи:  
x = filter_houses[idxs, :latitude];  
y = filter_houses[idxs, :longitude];  
scatter!(x, y)
```

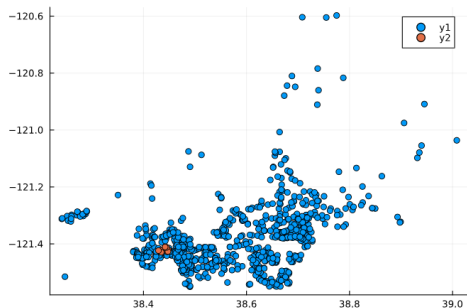


Рис. 19: Определение соседей объекта недвижимости

Обработка данных. Метод главных компонент

```
: # Фрейм с указанием площади и цены недвижимости:  
F = filter_houses[:, [:sq_ft, :price]]
```

814x2 DataFrame

Row	sq_ft	price
	Int64	Int64
1	836	59222
2	1167	68212
3	796	68880
4	852	69307
5	797	81900
6	1122	89921
7	1104	90895

```
: # Конвертация данных в массив:  
F = Matrix{Float64}(F)
```

```
: 814x2 Matrix{Float64}:  
 836.0  59222.0  
1167.0  68212.0  
 796.0  68880.0  
 852.0  69307.0  
 797.0  81900.0  
1122.0  89921.0  
1104.0  90895.0  
1177.0  91002.0  
 941.0  94905.0  
1146.0  98937.0  
 909.0 100309.0  
1289.0 106250.0  
 871.0 106852.0  
  ⋮  
 960.0 224252.0  
1456.0 225000.0  
1450.0 228000.0
```

Рис. 20: Создание фрейма

```
using MultivariateStats

# Приведение типов данных к распределению для PCA:
M = fit(PCA, F)

PCA(indim = 814, outdim = 1, principalratio = 1.0)

Pattern matrix (unstandardized loadings):
```

	PC1
1	41285.1
2	47488.0
3	48142.7
4	48405.0
5	57348.5
6	62790.4
7	63491.8
8	63515.9
9	66442.6
10	69148.7
11	70286.4
12	74218.6

```
# Выделение значений главных компонент в отдельную переменную:
#Y = MultivariateStats.transform(M, F)
Xr = reconstruct(M, y)
# Построение графика с выделением главных компонент:
scatter(F[1, :], F[2, :])
scatter!(Xr[1, :], Xr[2, :])
```

Рис. 21: Определение главных компонент для данных по объектам недвижимости

```
xvals = repeat(1:0.5:10,inner=2)
yvals = 3 .+ xvals + 2*rand(length(xvals)) .- 1
scatter(xvals,yvals,color=:black,leg=false)
```

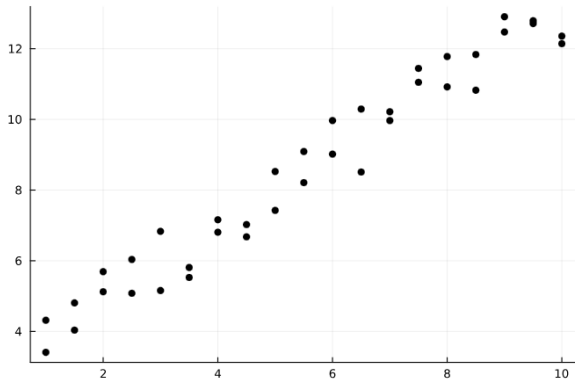


Рис. 22: Исходные данные

```
function find_best_fit(xvals,yvals)
    meanx = mean(xvals)
    meany = mean(yvals)
    stdx = std(xvals)
    stdy = std(yvals)
    r = cor(xvals,yvals)
    a = r*stdy/stdx
    b = meany - a*meanx
    return a,b
end
```

find_best_fit (generic function with 1 method)

```
a,b = find_best_fit(xvals,yvals)
ynew = a * xvals .+ b
plot!(xvals,ynew)
```

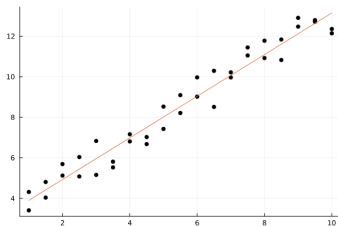


Рис. 23: Линейная регрессия

```
using PyCall
using Conda

py"""
import numpy
def find_best_fit_python(xvals,yvals):
    meanx = numpy.mean(xvals)
    meany = numpy.mean(yvals)
    stdx = numpy.std(xvals)
    stdy = numpy.std(yvals)
    r = numpy.corrcoef(xvals,yvals)[0][1]
    a = r*stdy/stdx
    b = meany - a*meanx
    return a,b
"""

xpy = PyObject(xvals)
ypy = PyObject(yvals)
@time a,b = py"find_best_fit_python($xvals, $yvals)"

0.199915 seconds (100.36 k allocations: 5.082 MiB, 35.74% compilation time)
(1.0000000134382265, 2.999466101573489)

using BenchmarkTools
@btime a,b = py"find_best_fit_python($xvals, $yvals)"
@btime a,b = find_best_fit(xvals,yvals)

5.614 ms (31 allocations: 768 bytes)
706.400 μs (1 allocation: 32 bytes)
(1.000000013438223, 2.999466101748112)
```

Рис. 24: Анализ производительности

```
using RDatasets  
iris = dataset("datasets", "iris")
```

150×5 DataFrame

Row	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
	Float64	Float64	Float64	Float64	Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

Рис. 25: Загрузка данных

Самостоятельная работа. Кластеризация

```
# Подготовка данных для кластеризации
# Берем только числовые признаки (столбцы 1:4) и транспонируем
iris_features = Matrix(iris[:, 1:4])'
println("Размерность признаков: ", size(iris_features))
```

Размерность признаков: (4, 150)

```
# Кластеризация методом k-средних
# Используем 3 кластера (так как в Iris 3 вида цветков)
k = 3
result = kmeans(iris_features, k)
```

```
KmeansResult{Matrix{Float64}, Float64, Int64}([5.005999999999999 6.853846153846153
7377; 1.4620000000000002 5.715384615384615 4.388524590163935; 0.2459999999999999 2
... 2, 2, 3, 2, 2, 2, 3, 2, 2, 3], [0.0199800000000018094, 0.20038000000000977, 0.17
00000534, 0.17238000000000397, 0.004380000000011819, 0.6519799999999947, 0.1415800
2621, 0.1127218934910843, 0.27272189349113773, 0.3557988165680399, 0.8221177102929
0, 39, 61], [50, 39, 61], [78.85566582597667, 4, true])
```

```
# Получаем метки кластеров
labels = result.assignments
println("Метки кластеров: ", unique(labels))
```

Метки кластеров: [1, 2, 3]

Рис. 26: Проведение кластеризации

Самостоятельная работа. Кластеризация

```
# Создаем точечную диаграмму
scatter(iris[:, :SepalLength], iris[:, :SepalWidth],
        group=iris[:, :Cluster],
        markersize=6,
        markerstrokewidth=0.5,
        xlabel="Длина чашелистика (Sepal Length)",
        ylabel="Ширина чашелистика (Sepal Width)",
        title="Кластеризация Iris методом k-средних",
        legend=:topright,
        palette=:rainbow)
```

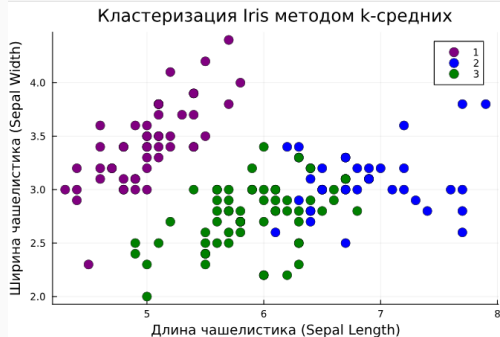


Рис. 27: Полученный график

Самостоятельная работа. Регрессия (метод наименьших квадратов в случае линейной регрессии)

```
# Часть I - данные
N = 1000
X = randn(N, 3)
a0 = rand(3)
y = X * a0 + 0.1 * randn(N)

1000-element Vector{Float64}:
-0.0776454544446147
 0.23606986662887048
-0.2019324522433486
-0.31894699745526145
-1.6627960860134292
-2.4499581046002046
-0.2638049000904011
-0.26222406506197593
 0.060724583222939576
-0.2849932602155379
-0.47146580690952677
-0.36947764494261004
 0.2872442958470945
 ⋮
 0.3847320661382492
-1.5392747752930063
 1.615273466473045

# II. Решение методом наименьших квадратов вручную
function manual_ols(X, y)
    # Добавляем столбец единиц для intercept (свободного члена)
    X_with_intercept = hcat(ones(size(X, 1)), X)

    # Решаем нормальное уравнение: (X'X)a = X'y
    a_manual = X_with_intercept \ y
    return a_manual
end

a_manual = manual_ols(X, y)
println("Ручное решение МНК: ", a_manual)

Ручное решение МНК: [0.0009383773719385517, 0.4475106797736989, 0.9943051796239224, 0.15251428656294774]
```

Рис. 28: Загрузка данных

Самостоятельная работа. Регрессия (метод наименьших квадратов в случае линейной регрессии)

```
# 2. Использование MultivariateStats.jl
using MultivariateStats

# llsq ожидает, что каждая строка - это наблюдение
model_llsq = llsq(X, y)
println("MultivariateStats (llsq): ", model_llsq)

MultivariateStats (llsq): [0.447510679773699, 0.9943051796239223, 0.15251428656294788, 0.000930377371938512]
```

```
# 3. Использование GLM.jl
using StatsModels, GLM
df = DataFrame(X1 = X[:,1], X2 = X[:,2], X3 = X[:,3], Y = y)
model_glm = lm(@formula(Y ~ X1 + X2 + X3), df)
println("GLM результат:")
println(model_glm)
```

GLM результат:
StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}, GLM.DensePredChol{Float64, LinearAlgebra.Cholesky{Matrix{Float64}}}}, Matrix{Float64}}

$Y \sim 1 + X1 + X2 + X3$

Coefficients:

	Coef.	Std. Error	t	Pr(> t)	Lower 95%	Upper 95%
(Intercept)	0.000930377	0.00311828	0.30	0.7655	-0.00518878	0.00704953
X1	0.447511	0.00312162	143.36	<1e-99	0.441385	0.453636
X2	0.994305	0.00313764	316.90	<1e-99	0.988148	1.00046
X3	0.152514	0.00313253	48.69	<1e-99	0.146367	0.158661

Рис. 29: Применение различных методов

Самостоятельная работа. Регрессия (метод наименьших квадратов в случае линейной регрессии)

```
# Сравнение с истинными значениями
println("\nСравнение с истинными параметрами:")
println("Истинные: [intercept, a1, a2, a3] ≈ [0, $(a0[1]), $(a0[2]), $(a0[3])]")
println("Ручной МНК: ", round(a_manual, digits=4))
println("MultivariateStats: ", round.(model_llsq, digits=4))
```

Сравнение с истинными параметрами:
Истинные: [intercept, a1, a2, a3] ≈ [0, 0.4530043496376218, 0.9923343625711946, 0.15170125746147456]
Ручной МНК: [0.0009, 0.4475, 0.9943, 0.1525]
MultivariateStats: [0.4475, 0.9943, 0.1525, 0.0009]

Рис. 30: Сравнение с истинными значениями

Самостоятельная работа. Регрессия (метод наименьших квадратов в случае линейной регрессии)

```
# Часть 2 - данные
X_simple = rand(100)
y_simple = 2 .* X_simple + 0.1 .* randn(100)

# Решение простой линейной регрессии
function simple_ols(X, y)
    X_with_ones = hcat(ones(length(X)), X)
    coefficients = X_with_ones \ y
    return coefficients
end

simple_ols (generic function with 1 method)

coeff = simple_ols(X_simple, y_simple)
intercept, slope = coeff[1], coeff[2]

println("Простая линейная регрессия:")
println("Intercept: ", round(intercept, digits=4))
println("Slope: ", round(slope, digits=4))

Простая линейная регрессия:
Intercept: 0.0002
Slope: 1.9804
```

Рис. 31: Загрузка данных

Самостоятельная работа. Регрессия (метод наименьших квадратов в случае линейной регрессии)

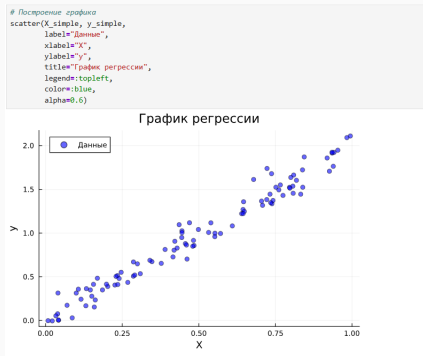


Рис. 32: Построение графика

Самостоятельная работа. Регрессия (метод наименьших квадратов в случае линейной регрессии)

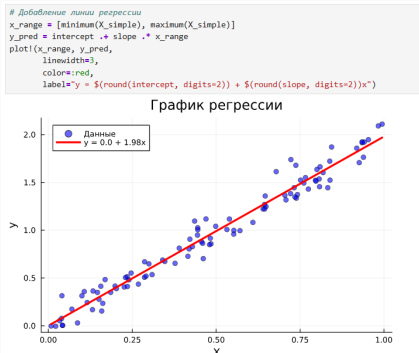


Рис. 33: График регрессии



Рис. 34: Биномиальная модель: траектория цены акции

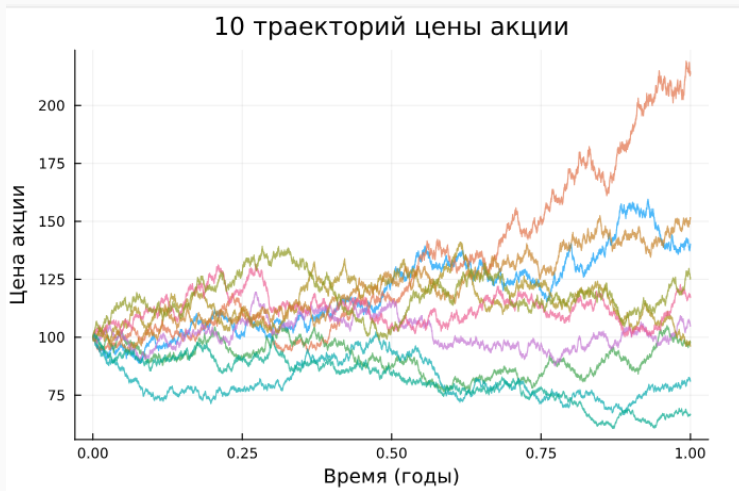


Рис. 35: Построение 10 траекторий цены акции

```
# c) Распараллеливание генерации траекторий

# Способ 1: Использование Threads.@threads
using Base.Threads

function generate_paths_threads(num_paths::Int, S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    paths = Vector{Vector{Float64}}(undef, num_paths)

    @threads for i in 1:num_paths
        paths[i] = createPath(S, r, sigma, T, n)
    end

    return paths
end

# Тестирование распараллеленных версий
println("Генерация 100 траекторий с использованием Threads...")
paths_threads = generate_paths_threads(100, S, r, sigma, T, n)

Генерация 100 траекторий с использованием Threads...
100-element Vector{Vector{Float64}}:
```

Рис. 36: Генерация 100 траекторий с использованием Threads

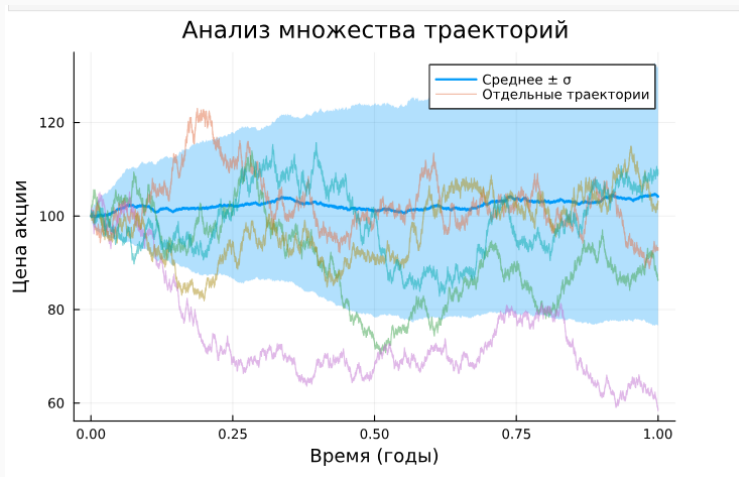


Рис. 37: Анализ множества траекторий

Выводы

Основной целью работы является специализированных пакетов Julia для обработки данных.